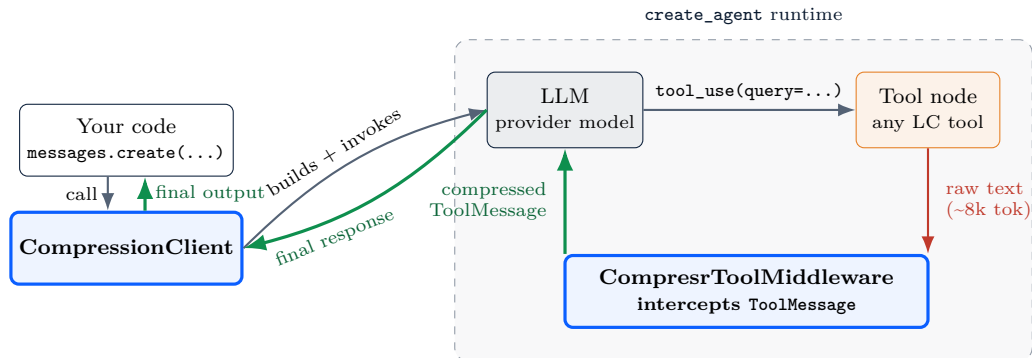


Compresr SDK — LangChain/LangGraph Integration & the Web Search Tool

1 Architecture

The whole SDK is one substitution: replace your provider client with `CompressionClient` and the same call-site code (`client.messages.create(...)`) keeps working — except every tool result now passes through Compresr middleware before reaching the LLM.



2 Part 1 — Wrapping any LangChain / LangGraph tool

If you have a LangChain tool, you already have a Compresr-compressible tool. No subclassing, no decoration. Three integration paths:

2.1 Path A — The facade (default, 90% of use cases)

Pass tools to `client.messages.create(...)` / `client.chat.completions.create(...)`; the middleware is wired automatically. Any LangChain tool works — here we just pass a `WebSearchTool`.

```
from compresr import CompressionClient, WebSearchTool

client = CompressionClient(
    api_key=COMPRESR_KEY,
    llm="anthropic", llm_api_key=ANTHROPIC_KEY,
    compression={"target_compression_ratio": 0.5, "min_tokens": 300},
)

response = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=512,
    messages=[{"role": "user",
                "content": "Analyse the competitive landscape for X"}],
    tools=[WebSearchTool.brave(api_key=BRAVE_KEY, max_results=3)],
)
```

The tool flows through `CompresrToolMiddleware.wrap_tool_call` and gets compressed automatically when its text output exceeds `min_tokens`. You can also pass any custom `@tool`-decorated function the same way — the middleware doesn't care what the tool does, only that its output is a string.

2.2 Path B — Explicit `create_agent`

Use this when you need to control the agent yourself — custom state, custom middleware stacks, or other LangChain features the facade doesn't expose. The pattern is: build a chat model with `init_chat_model`, call `create_agent` directly, and put `CompresrToolMiddleware` in the middleware list.

```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from compresr.integrations.langchain import CompresrToolMiddleware

chat = init_chat_model("anthropic:claude-sonnet-4-6", api_key=ANTHROPIC_KEY)
agent = create_agent(
    model=chat,
    tools=[my_tool_1, my_tool_2, web_search],
    middleware=[CompresrToolMiddleware(
        api_key=COMPRESR_KEY,
        target_compression_ratio=0.5, min_tokens=300,
        query_arg="query", # use tool arg "query" as the query
        allow_tools={"web_search"}, # only compress these tools
    )],
)

# invoke the LangGraph-style agent
result = agent.invoke({"messages": [{"role": "user", "content": "..."}]})
final_text = result["messages"][-1].content
```

How to use it:

1. `init_chat_model("anthropic:claude-sonnet-4-6", ...)` returns a `LangChain BaseChatModel`.
2. `create_agent(model, tools, middleware)` compiles a `LangGraph`-style `ReAct` agent.
3. The `middleware` list is just normal `LangChain` middleware — you can chain `CompresrToolMiddleware` with your own (e.g. logging, guardrails, summarisation).
4. `agent.invoke({"messages": [...]})` runs the loop; the final assistant text is at `result["messages"][-1].content`.
5. Use `allow_tools` / `ignore_tools` to scope compression to a subset of tools, and `query_arg` to tell `latte_v1` which tool argument is the query.

Query resolution — one rule, five sources

The compression engine is query-aware — what gets kept depends on what you asked. The SDK resolves the query from these sources, in priority order:

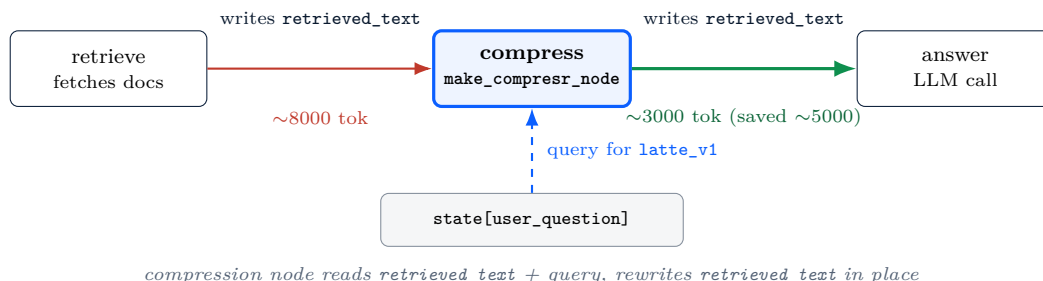
1. `query="..."` — static, same for every call.
2. `query_extractor=fn(call,msgs)` — custom callable.
3. `query_arg="name"` — read the tool's own argument with that name.
4. Smart-pick from common keys (`query`, `question`, `search_query`, ...).
5. Fallback: the last `HumanMessage` in history.

2.3 Path C — LangGraph node (non-agent pipelines)

Use this when there is no tool-calling loop at all — a plain RAG pipeline, a multi-step state graph, etc. `make_compresr_node` returns a node function that reads a state field, compresses it in place, and writes it back. Drop it anywhere in your `StateGraph`.

```
from langgraph.graph import StateGraph
from compresr.integrations.langgraph import make_compresr_node

graph = StateGraph(MyState)
graph.add_node("retrieve", retrieve_docs)
graph.add_node("compress", make_compresr_node(
    api_key=COMPRESR_KEY,
    context_key="retrieved_text", # read+write this state key
    query_key="user_question",    # use this key as the query
    target_compression_ratio=0.6,
))
graph.add_node("answer", answer_with_llm)
graph.add_edge("retrieve", "compress"); graph.add_edge("compress", "answer")
```



3 Part 2 — The web search tool

3.1 Why we need a web search tool at all

LLMs hallucinate when asked about anything outside training cut-off or anything narrow (specific companies, recent news, niche prices). For analyst-style prompts (“analyse the competitive landscape for X”) there are only two correct outcomes without grounding: refusal (useless) or fabrication (worse). A web search tool fixes that by letting the model fetch live evidence at runtime.

Three jobs the tool performs: (1) *Recency* — reach content created after the model’s training cut-off; (2) *Verifiability* — return citable URLs users can audit; (3) *Specificity* — surface long-tail facts the model never memorised.

Why it matters *specifically* for Compresr. A search call typically returns 3–10 hits with titles, URLs, and snippets/extracts — 2–10k tokens per call — and that text feeds every subsequent turn. This is the single biggest source of token bloat in agent loops, and it is exactly what the SDK exists to remove:

Raw web search		≈ 8500 tok
Compresr ratio=0.5		≈ 4200 tok
Compresr ratio=0.7		≈ 2500 tok

The web search tool and the compression layer are designed to be used *together*.

3.2 What WebSearchTool actually is

A thin factory returning a regular LangChain BaseTool backed by either:

- **Tavily** (default) — via `langchain_tavily.TavilySearch`. Supports `allowed_domains/blocked_domains`.
- **Brave** — via `langchain_community.tools.BraveSearch`, wrapped in a `StructuredTool` with an explicit pydantic query schema (without it, Anthropic tool-use emits an empty args dict because upstream Brave ships with `args_schema=None`).

```
search = WebSearchTool.tavily(api_key=TAVILY_KEY, max_results=5,
                             allowed_domains=["nytimes.com", "reuters.com"])
# OR
search = WebSearchTool.brave(api_key=BRAVE_KEY, max_results=5)

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=512,
    messages=[{"role": "user", "content": "Analyze competitive landscape for X"}],
    tools=[search],
)
```

3.3 Difference vs. Anthropic’s web_search_20250305

Anthropic’s [server-side web search](#) (type: `"web_search_20250305"`) and the Compresr `WebSearchTool` look alike — both let Claude search the web — but architecturally they are different things. **Anthropic’s is not a chain in your process: it is a black box behind the API.** The model emits a `server_tool_use` block, Anthropic’s infrastructure runs the search, and you get back opaque `web_search_tool_result` blocks with citation URLs — but the underlying hit content never enters your process in raw form. You cannot intercept it; therefore Compresr cannot compress it. Compresr’s tool runs *client-side*, so the result lands in your process as plaintext, and the middleware can read and shrink it.

	Anthropic <code>web_search_20250305</code>	Compresr <code>WebSearchTool</code>
Where search runs	Anthropic servers (opaque)	Your process (LangChain tool)
Result visibility	Encrypted / not in your process	Plaintext, inspectable
Compressible by Compresr	No — nothing to intercept	Yes — the design goal
Search backends	Anthropic’s curated index	Tavily, Brave (pluggable)
Billing	Per-search via Anthropic	Direct to Tavily / Brave
Domain filtering	Limited	<code>allowed/blocked_domains</code> (Tavily)
Cross-provider	Anthropic only	Same code on Claude / GPT / Gemini
Logging / auditing	Not possible	Full visibility in your process

Why we forbid the server-side tool. It is not a chain we can sit in — it is a sealed call behind the API. Tools and middleware live in your process; the server tool’s payload never gets there. With no interception point, there is no compression. Use Tavily or Brave and the middleware can do its job.

3.4 Migration from `robust_client.call_with_retry`

```
# BEFORE -- server-side tool, opaque to your code
tools=[{"type": "web_search_20250305", "name": "web_search",
        "max_uses": WEB_SEARCH_MAX_USES}]

# AFTER -- client-side, compressible
from compresr import WebSearchTool
tools=[WebSearchTool.brave(api_key=BRAVE_KEY, max_results=3)]
# or: WebSearchTool.tavily(api_key=TAVILY_KEY, max_results=3)
```

The wrapping `RobustClient` is unchanged. It still calls `self.client.messages.create(**request_params)`; what changed is that `self.client` is now `CompressionClient` and the tool produces compressible plaintext instead of opaque server-tool output.

3.5 Getting the final result — three surfaces, same engine

Compresr exposes three equivalent surfaces on the same client. Pick by ergonomics; behaviour and middleware path are identical. The model lives at the *call site*, just like Anthropic’s and OpenAI’s own SDKs.

1. Anthropic shape — response is an `anthropic.types.Message`; iterate `response.content` for text and `tool_use` blocks (matches the existing `RobustClient` code path):

```
resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=512,
    messages=[{"role": "user", "content": "Analyze competitor landscape for X"}],
    tools=[WebSearchTool.brave(api_key=BRAVE_KEY, max_results=3)],
)
text = "".join(b.text for b in resp.content if hasattr(b, "text"))
stop = resp.stop_reason          # "end_turn" / "tool_use" / "max_tokens"
usage = resp.usage               # input_tokens, output_tokens, ...
```

2. OpenAI shape — response is the OpenAI `ChatCompletion`; read `choices[0].message.content`:

```

client_oai = CompressionClient(
    api_key=COMPRESR_KEY,
    llm="openai", llm_api_key=OPENAI_KEY,
    compression={"target_compression_ratio": 0.5},
)
resp = client_oai.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Analyse competitor landscape for X"}],
    tools=[WebSearchTool.tavily(api_key=TAVILY_KEY, max_results=5)],
)
text = resp.choices[0].message.content
finish = resp.choices[0].finish_reason

```

3. Native `client.run(...)` — provider-agnostic. Returns a `NormalizedResult` with the same fields no matter which provider you target. Best for code that should work across Claude / GPT / Gemini:

```

result = client.run(
    prompt="Analyse competitor landscape for X",
    model="claude-sonnet-4-6", # or "gpt-4o-mini" / "gemini-1.5-pro" with right llm=
    tools=[WebSearchTool.brave(api_key=BRAVE_KEY, max_results=3)],
    max_tokens=512, temperature=0.2,
)
result.text           # str -- final assistant text
result.tool_uses      # list -- [{id, name, input}, ...]
result.citations      # list -- [Citation(url, title, cited_text), ...] normalised
result.stop_reason    # str -- "end_turn" / "tool_use" / ...
result.usage          # dict -- {input_tokens, output_tokens, ...}
result.raw            # the underlying provider message (escape hatch)

```

Per-call LLM knobs. `temperature`, `top_p`, `stop_sequences`, `seed`, `presence_penalty`, etc. pass through to the underlying chat model via `.bind(...)` on each call — the cached model is never mutated, so switching knobs between calls is safe.